설비 고장을 예측하는 AI 데이터 분석

추세·주기·변동성 해석

변화량과 주기 분석

posco

CONTENTS

목차

01 변화량 분석 — diff·급변 탐지·종합 관찰

02 주기와 시간 기반 분석 — 시간 인덱스·리샘플링

03 누락과 분해 — 보간·시계열 분해

posco

01

변화량 분석

diff·pct_change로 급변 시점 포착, 추세·변동성과의 종합 관찰까지

posco

변화량이란 무엇인가

추세가 멀리서 본 큰 흐름이라면, 변화량은 매 순간의 발걸음

변화량은 한 시점 값에서 **바로 앞 시점 값**을 뺀 차이.

추세가 멀리서 본 큰 흐름이라면, 변화량은 매 순간의 발걸음.

고장은 종종 **급격한 변화**로 나타나 변화량으로 잘 잡힘.

예시 — 어제 70kg, 오늘 69.5kg이면 변화량은 −0.5kg.
값 자체가 아니라 값의 **움직임**에 주목하는 관점.

절대 변화량과 변화율의 차이

같은 5라도 절대 5와 5퍼센트는 다른 이야기, 크기 다른 센서를 견줄 땐 비율

01 · 절대 변화량

변한 양 그대로

100 → 105이면

+5 그대로

실제로 얼마나 움직였나가 중요할 때 (예: 온도 1℃ 상승)

pandas **.diff()**로 계산

02 · 변화율

원래 값 대비 비율

100 → 105이면

+5%

크기 다른 센서를 공정 비교할 때 (10에서 2 변한 건 20%, 1000에서 2는 0.2%)

pandas **.pct_change()**로 계산

절대 변화량이 같아도 변화율은 완전히 다를 수 있음 · 비교가 필요할 땐 비율이 공정한 잣대

변화량이 급변 포착에 좋은 이유

원본에선 큰 값에 묻히는 1℃가, 변화량에선 우뚝 솟는다

01 · 평소엔 잠잠

평소엔 **0 근처**에 머물다가 급변 시점에만 크게 튐

02 · 묻히던 변화가 도드라짐

원본의 큰 값에 묻히던 변화가 변화량에선 **도드라짐**

03 · 단, 주의

노이즈에 민감해 진짜 급변과 구분이 필요

643℃ 원본에선 1℃ 변화가 묻히지만, 변화량 그래프에선 그 1℃가 우뚝 솟음

posco

변화량 구하기 — diff / pct_change

.diff()는 변한 양, .pct_change()는 변한 비율

CODE

```
s = pd.Series([100, 103, 102, 108, 107])

print(s.diff().tolist())
# [nan, 3.0, -1.0, 6.0, -1.0]

print(s.pct_change().round(4).tolist())
# [nan, 0.03, -0.0097, 0.0588, -0.0093]
```

첫 값이 NaN인 건 변화량의 자연스러운 성질이지 오류가 아님

posco

실습 1. 변화량으로 급변 탐지하기

원본에서 안 보이던 급변 시점을 변화량으로 찾기

목표

직전 대비 변화량으로 원본에서 안 보이던 급변 시점을 찾기

단계

· 2번 엔진 센서의 직전 대비 변화량을 구하기

· 위칸에 원본, 아래칸에 변화량과 0 기준선을 그리기

· 0에서 크게 벗어난 회차를 급변 시점으로 표시

예상 결과

변화량 그래프에서 0을 크게 벗어난 몇몇 회차가 급변 후보

posco

실습 1. 변화량으로 급변 탐지하기 정답

원본에서 안 보이던 급변 시점을 변화량으로 찾기

CODE · 실습 1

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv('cmapss_fd001_sample.csv')
e2 = df[df['unit_nr'] == 2].reset_index(drop=True)
d = e2['s_11'].diff()
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(11, 6), sharex=True)
ax1.plot(e2['time_cycles'], e2['s_11']); ax1.set_title('s_11 raw')
ax2.plot(e2['time_cycles'], d, color='green'); ax2.axhline(0, color='gray')
ax2.set_title('s_11 diff'); plt.tight_layout(); plt.show()
```

posco

급변 시점 해석 시 주의점

큰 변화량은 주목 신호이지 고장 확정이 아님

01 · 노이즈

잡음 출렁임

우연히 한 시점이 크게 튀면
변화량도 따라 튐

진짜 변화가 아니라 잡음

이동평균 후 변화량으로 **평활** 권장

02 · 운영 변화

정상 반응

가동·정지 / 부하 변경 시
정상적으로 크게 변함

멈췄다 켜거나 부하를 높이면 정상 반응

운영 기록 **교차 확인**

03 · 측정 오류

센서 / 누락

오작동 / 데이터 누락 경계에서
값이 튀어 보임

센서 오작동이나 NaN 복구 시점에서 발생

누락 점검 **습관화**

원본 값, 다른 센서, 운영 기록을 함께 보고 종합 판단

posco

실습 2. 추세·변동성·변화량 종합 관찰

한 센서를 세 관점으로 한 화면에서 관찰

목표

한 센서를 추세·변동성·변화량 세 관점으로 한 화면에서 관찰

단계

· 이동평균으로 추세, 이동표준편차로 변동성, 변화량으로 급변을 각각 구하기

· 세 칸으로 나눠 원본과 추세, 변동성, 변화량을 그리기

· 세 관점을 종합해 센서 상태를 한두 문장으로 정리

예상 결과

추세는 완만한 상승, 변동성·변화량은 후반에 커지는 신호

posco

실습 2. 추세·변동성·변화량 종합 관찰 정답

한 센서를 세 관점으로 한 화면에서 관찰

CODE · 실습 2

```
col = 's_3'
ma = e2[col].rolling(20).mean(); sd = e2[col].rolling(20).std(); d = e2[col].diff()
fig, axes = plt.subplots(3, 1, figsize=(11, 8), sharex=True)
axes[0].plot(e2['time_cycles'], e2[col], alpha=0.4); axes[0].plot(e2['time_cycles'], ma)
axes[0].set_title('value & MA20 (trend)')
axes[1].plot(e2['time_cycles'], sd, color='red'); axes[1].set_title('moving std')
axes[2].plot(e2['time_cycles'], d, color='green'); axes[2].axhline(0, color='gray')
axes[2].set_title('diff'); plt.tight_layout(); plt.show()
```

02

주기와 시간 기반 분석

주기성 개념 · 시간 인덱스 · 리샘플링으로 패턴 드러내기

주기성이란 무엇인가

추세는 한 방향, 주기성은 같은 모양의 되풀이

주기성은 일정한 간격으로 **비슷한 패턴이 반복**되는 성질.

추세가 한 방향이라면, 주기성은 같은 모양의 되풀이.

하루 기온, 평일·주말 전력처럼 일정 간격으로 반복.

설비도 가동·정지를 반복하면 센서값에 주기가 생김.
이 **정상 주기**가 흐트러지면 이상 신호.

설비와 생활 속 주기 예시

주기는 정상의 리듬, 평소 리듬이 깨지는 순간이 이상 신호

01 · 하루 주기

24시간 단위

기온·전력·교통량이
24시간마다 반복

낮·밤 사이클이 만드는 가장 익숙한 주기

period = **24h**

02 · 일주일 주기

7일 단위

평일과 주말의 차이가
7일마다 반복

근무·휴일 리듬이 만드는 주기

period = **7d**

03 · 설비 주기

가동·정지 간격

반복 작업 간격이
곧 설비의 주기

가열·냉각, 낮 가동·밤 정지가 만드는 주기

설비별 **고유 주기** 파악

이 데이터에 어떤 주기가 있나, 익숙한 예에서 출발해 생각

주기 길이 추정 개념

한 봉우리에서 다음 봉우리까지의 거리가 곧 주기 길이

주기 길이는 같은 패턴이 한 번 반복되는 데 걸리는 간격.

그래프에서 **봉우리 사이 간격**으로 가늠.

입문 단계에선 눈으로 재는 것으로 충분, 복잡한 수학 불필요.

> **정확한 숫자보다 감각이 먼저** — 이 데이터에 주기가 있는가,
> 대략 어느 간격인가를 파악하는 능력.

posco

cycle 기반과 timestamp 기반의 차이

cycle은 몇 번째인가, timestamp는 언제인가

01 · CYCLE 기반

순서 번호만 있음

```
cycle: 1, 2, 3, ...
```

가능 — 추세·변동성·변화량
불가 — 시간 단위 분석

C-MAPSS 원본 — 첫 시간에서 사용

02 · TIMESTAMP 기반

실제 시각 있음

```
2024-01-01 09:00, ...
```

가능 — 하루 평균·주말 비교
가능 — 시간 단위 묶기·자르기

가상 시각 샘플 — 이번 시간에서 사용

C-MAPSS엔 시각이 없어 가동 주기를 가상 시각으로 바꾼 샘플을 별도 준비

시간 인덱스가 필요한 이유

시각을 인덱스로 만들면 시간 기반 기능이 열림

01 · 시각을 이름표로

시각을 **인덱스**로 만들면 시간 기반 기능이 열림

02 · 시간 단위로 묶기

하루 평균, 시간별 최댓값을 **한 줄로**

03 · 특정 기간으로 자르기

1월 3일만, 오전만 같은 식으로 **기간 슬라이싱**

촘촘한 데이터를 시간 단위로 묶어 평균 내면 노이즈 감소, 큰 흐름이 드러남

posco

시간 인덱스 적용 — to_datetime + set_index

글자 → 시각, 시각 → 인덱스, 두 단계로 시각 인덱스 만들기

CODE

```
ts = pd.read_csv('data/engine1_timestamp_sample.csv')
ts['timestamp'] = pd.to_datetime(ts['timestamp'])
ts = ts.set_index('timestamp')

print(type(ts.index).__name__)
# DatetimeIndex
```

DatetimeIndex로 나오면 pandas가 시간으로 이해한다는 표시 — 시간 분석 준비 완료

실습 3. 시간 단위 추세 보기

가로축을 진짜 날짜·시각으로 만들어 추세 보기

목표

가로축을 진짜 날짜·시각으로 만들어 원본과 이동평균 추세를 보기

단계

· 시각 글자를 시각 값으로 바꿔 인덱스로 올리기

· 원본을 옅게, 시간 이동평균을 진하게 겹쳐 그리기

· 가로축이 실제 시각인 추세선을 읽기

예상 결과

날짜 가로축 위로 매끈하게 상승하는 이동평균선

posco

실습 3. 시간 단위 추세 보기 정답

가로축을 진짜 날짜·시각으로 만들어 추세 보기

CODE · 실습 3

```
ts = pd.read_csv('engine1_timestamp_sample.csv')
ts['timestamp'] = pd.to_datetime(ts['timestamp'])
ts = ts.set_index('timestamp')
plt.plot(ts.index, ts['s_4'], alpha=0.4, label='raw')
plt.plot(ts.index, ts['s_4'].rolling(10).mean(), label='MA(10)')
plt.title('s_4 over time'); plt.legend(); plt.show()
```

리샘플링이 패턴을 드러내는 원리

묶어서 평균 내면 노이즈는 상쇄, 큰 패턴은 드러남

리샘플링은 데이터를 다른 **시간 단위로 다시 묶는** 것.

묶어서 평균 내면 노이즈가 상쇄되고 큰 패턴이 드러남.

이동평균과 달리 **데이터 개수 자체가 줄어듦**.

예시 — 1시간 간격 192개를 하루로 묶으면 8개로 줄어듦.
641.4 → 646.6까지 매끈하게 오르는 흐름이 드러남.

posco

resample로 시간 단위 집계 적용

.resample()로 묶을 단위, .mean()으로 집계 함수

CODE

```
daily = ts['s_2'].resample('D').mean()

print(daily.round(2).tolist())
# [641.4, 642.23, 642.8, 643.68,
# 644.34, 644.97, 645.78, 646.63]
```

'D' 하루 · '6h' 6시간 · 'h' 1시간 · 집계 함수는 mean·max 등 자유 선택

실습 4. 시간 단위 집계로 주기 패턴 보기

원본·6시간·하루 평균을 겹쳐 리샘플링 효과 확인

목표

원본·6시간·하루 평균을 겹쳐 리샘플링이 큰 패턴을 드러냄을 확인

단계

· 시각 인덱스 데이터에서 6시간·하루 단위 평균을 각각 구하기

· 원본과 두 집계를 한 그래프에 겹쳐 그리기

· 단위가 커질수록 잔잔해지며 큰 흐름이 드러남을 확인

예상 결과

하루 평균은 8개 점으로, 원본의 잔떨림이 사라진 매끈한 흐름

posco

실습 4. 시간 단위 집계로 주기 패턴 보기 정답

원본·6시간·하루 평균을 겹쳐 리샘플링 효과 확인

CODE · 실습 4

```
h6 = ts['s_4'].resample('6h').mean()
day = ts['s_4'].resample('D').mean()
plt.plot(ts.index, ts['s_4'], alpha=0.3, label='raw(1h)')
plt.plot(h6.index, h6, marker='o', label='6h mean')
plt.plot(day.index, day, marker='s', label='daily mean')
plt.legend(); plt.show()
print('하루 평균 개수:', len(day))     # 8
```

시간 단위 선택이 해석을 바꾼다

한 단위만 믿지 말고 여러 단위로 번갈아 확인

같은 데이터도 묶는 단위에 따라**보이는 패턴이 달라짐**.

너무 크게 묶으면 짧은 주기가 사라지고, 너무 작으면 노이즈가 남음.

보고 싶은 패턴보다 **작은 단위**로 묶어야 그 패턴이 살아남음.

s_2 하루 평균은 매끈한 상승이지만, 1시간 원본으로 보면 하루 안에서 출렁임.
결과를 보고할 땐 **어떤 단위로 봤는지 함께 밝힘**.

03

누락과 분해

NaN 점검·보간 방법 선택·시계열 분해의 결과 읽기

posco

측정 주기 불일치와 누락 시점 문제

분석 전 .isna().sum()으로 누락 개수 점검을 습관화

현실 데이터는 센서 고장·통신 끊김으로 값이 **누락(NaN)**될 수 있음.

누락이 있으면 이동평균·변화량 계산이 어긋남.

분석 전 **isna로 누락 개수를 점검**하는 습관 필요.

변화량은 직전 값이 없어 엉뚱하게, 이동평균은 일부 값만으로 계산됨.
누락 점검이 모든 시계열 분석의 출발점.

posco

보간 방법 선택 기준

보간값은 진짜가 아니라 추정, 데이터 성격에 맞춰 방법 선택

방법	채우는 값	적합한 경우
`.ffill()`	앞의 값	잘 안 변하는 값 (설비 상태값 등)
`.bfill()`	뒤의 값	앞이 누락일 때
`.interpolate()`	앞뒤 직선값	꾸준히 변하는 값 (온도 등)

짧은 누락엔 보간, 긴 누락엔 제외가 나음 — 채운 값을 측정값처럼 맹신 금지

실습 5. 누락 시점 보간 후 재분석

채우는 방법에 따라 결과가 어떻게 다른지 비교

목표

누락을 만들고 채우는 방법에 따라 결과가 어떻게 다른지 비교

단계

· 작은 예시에서 앞값 채움과 선형 보간의 결과 차이를 비교

· 시각 데이터 일부를 일부러 비우고 누락 개수를 확인

· 보간으로 채운 뒤 누락이 사라졌는지 확인

예상 결과

앞값 채움은 계단 모양, 선형 보간은 12·14로 이어짐; 보간 후 0

posco

실습 5. 누락 시점 보간 후 재분석 정답

채우는 방법에 따라 결과가 어떻게 다른지 비교

CODE · 실습 5

```
import numpy as np
s = pd.Series([10.0, np.nan, np.nan, 16.0, 18.0])
print('ffill:', s.ffill().tolist())            # [10, 10, 10, 16, 18]
print('interpolate:', s.interpolate().tolist())  # [10, 12, 14, 16, 18]
s2 = ts['s_4'].copy(); s2.iloc[50:55] = np.nan
print('누락 개수:', int(s2.isna().sum()))         # 5
print('보간 후:', int(s2.interpolate().isna().sum()))  # 0
```

주기와 추세를 함께 볼 때 주의점

보고 싶은 것에 맞춰 다른 것을 걷어내기

실제 데이터엔 추세와 주기가 **동시에** 들어 있는 경우가 많음.

섞이면 한쪽이 다른 쪽을 가려 해석이 헷갈림.

추세를 볼 땐 큰 단위로 묶어 **주기를 평균으로 없앰**.

> 전력 사용량은 **하루 주기 + 여름철 추세 상승**이 겹쳐 그래프가 복잡.
> 한 번에 다 보려 하지 말고, 하루 평균으로 묶으면 날짜별 추세만 남음.

posco

시계열 분해 개념 (선택)

이런 것도 있다 — 입문 단계에선 결과 읽기까지가 목표

01 · 세 요소로 자동 분리

데이터를 **추세·주기·나머지** 세 요소로 자동 분리

02 · 이동평균과 같은 개념

추세 성분은 **이동평균으로 본 추세와 본질적으로 같음**

03 · 입문 단계의 목표

직접 만들기보다 **결과 읽기**까지가 목표

C-MAPSS는 뚜렷한 주기가 약해 분해 효과가 크진 않지만, 도구 작동 방식을 알아두면 좋음

posco

실습 6. 시계열 분해 시연·관찰

원본·추세·주기·나머지 네 부분으로 분해해 관찰

목표

시계열을 원본·추세·주기·나머지 네 부분으로 분해해 관찰(강사 시연)

단계

· 분해 도구로 시각 데이터를 더하기 모형으로 분해

· 원본·추세·계절·잔차 네 칸을 함께 보기

· 추세 칸이 이동평균 추세와 비슷한지 비교

예상 결과

네 칸으로 나뉜 분해 결과 — 추세는 상승, 주기는 반복 무늬

posco

실습 6. 시계열 분해 시연·관찰 정답

원본·추세·주기·나머지 네 부분으로 분해해 관찰

CODE · 실습 6

```
from statsmodels.tsa.seasonal import seasonal_decompose
result = seasonal_decompose(ts['s_4'], model='additive', period=24)
result.plot()
plt.tight_layout(); plt.show()
# 원본·추세(trend)·계절(seasonal)·잔차(resid) 4단
```